Machine Translated by Google

KHÁC

Sự khác biệt chính giữa HTTP=1.0 và HTTP=1.1

Balachander Krishnamurthy a,Ł, Jeffrey C. Mogul b, David M. Kristol c

[a] AT&T Labs-Research, 180 Park Avenue, Florham Park, NJ 07932, USA Western
[b] Research Lab, Compaq Computer Corp., 250 University Avenue, Palo Alto, CA 94301, USA c Bell Laboratories,
600 Mountain Avenue, Murray Hill, NJ 07974 , Hoa Kỳ

trừu tượng

Giao thức HTTP=1.1 là kết quả của bốn năm thảo luận và tranh luận giữa một nhóm lớn các nhà nghiên cứu Web và các nhà phát triển. Nó cải thiện dựa trên thành công phi thường của người tiền nhiệm, HTTP=1.0, theo nhiều cách. Chúng tôi thảo luận về sự khác biệt giữa HTTP=1.0 và HTTP=1.1, cũng như một số lý do căn bản đằng sau những thay đổi này. 1999 Xuất bản bởi Elsevier Science BV Mọi quyền được bảo lưu.

Từ khóa: HTTP=1.0; HTTP=1.1

1. Giới thiệu

Theo bất kỳ tiêu chuẩn hợp lý nào, giao thức HTTP=1.0 đã thành công một cách đáng ngạc nhiên. Là một biện pháp về mức độ phổ biến của nó, HTTP chiếm khoảng 75% Lưu lượng truy cập đường trục Internet trong một nghiên cứu gần đây [35]. TRONG Tuy nhiên, mặc dù thành công của nó, HTTP=1.0 vẫn được sử dụng rộng rãi hiểu là có nhiều sai sót.

HTTP=1.0 được phát triển từ phiên bản '0.9' ban đầu của HTTP (vẫn hiếm được sử dụng). Quá trình dẫn đến HTTP=1.0 liên quan đến cuộc tranh luận quan trọng và thử nghiệm, nhưng chưa bao giờ đưa ra thông số kỹ thuật chính thức. Nhóm làm việc HTTP (HTTP-WG) của Lực lượng Đặc nhiệm Kỹ thuật Internet (IETF) đã đưa ra một tài liệu (RFC1945) [2] mô tả 'cách sử dụng phổ biến' HTTP=1.0, nhưng chưa thử để tạo ra một tiêu chuẩn chính thức từ nhiều biến thể triển khai. Thay vào đó, trong khoảng thời gian khoảng bốn năm, HTTP-WG đã phát triển một cải tiến giao thức, được gọi là HTTP=1.1. Thông số HTTP=1.1 ification [9] sẽ sớm trở thành Tiêu chuẩn Dự thảo của IETF. Phiên bản gần đây của một số đại lý phổ biến (MSIE, Apache) yêu cầu tuân thủ HTTP=1.1 trong các yêu cầu hoặc phản hồi lại của họ và nhiều triển khai đã đã được thử nghiệm về khả năng tương tác tuân thủ với đặc điểm kỹ thuật [24,30].

Đặc tả HTTP=1.1 nêu rõ các yêu cầu khác nhau đối với máy khách, proxy và máy chủ. Tuy nhiên, bối cảnh và lý do bổ sung cho các tính năng mới hoặc đã thay đổi có thể giúp nhà phát triển hiểu được động lực đằng sau những thay đổi đó và cung cấp họ với sự hiểu biết phong phú hơn về giao thức. Ngoài ra, những lý do cơ bản này có thể cung cấp cho người thực hiện một cảm nhận rộng hơn về những ưu và nhược điểm của cá nhân đặc trưng.

Trong bài viết này, chúng tôi mô tả những thay đổi chính giữa các giao thức HTTP=1.0 và HTTP=1.1. Các Đặc tả HTTP=1.1 dài gần gấp ba lần như RFC1945, phản ánh sự gia tăng về độ phức tạp, sự rõ ràng, cụ thể. Mặc dù vậy, có rất nhiều quy định ngụ ý bởi đặc tả HTTP=1.1, thay vì được nêu rõ ràng. Trong khi một số nỗ lực có

Ł Tác giả tương ứng. Email: bala@research.att.com

1999 Được xuất bản bởi Elsevier Science BV Mọi quyền được bảo lưu.

Machine Translated by Google

được thực hiện để ghi lại sự khác biệt giữa HTTP=1.0 và HTTP=1.1 ([23,36], phần 19.6.1 của [9]), chúng tôi biết rằng không có phân tích nào được công bố đề cập đến những khác biệt lớn và lý do căn bản đằng sau chúng, đồng thời điều đó phản ánh bản sửa đổi gần đây nhất (và có thể là gần cuối cùng) của đặc tả HTTP=1.1. Bởi vì HTTP-WG, một nhóm lớn gồm các nhà nghiên cứu và phát triển quốc tế, đã tiến hành hầu hết các cuộc thảo luận thông qua danh sách gửi thư của mình, kho lưu trữ danh sách đó [5] ghi lại lịch sử của nỗ lực HTTP=1.1. Nhưng kho lưu trữ đó chứa hơn 8500 tin nhắn, khiến tất cả mọi người đều không rõ ràng, trừ người sử dụng giao thức kiên quyết nhất.

Chúng tôi cấu trúc cuộc thảo luận của mình bằng cách (hơi tùy tiện) chia các thay đổi về giao thức thành chín chính khu vực:

(1) Khả năng mở rộng
(2) Bộ nhớ đệm
(3) Tối ưu hóa băng thông
(4) Quản lý kết nối mạng
(5) Truyền tin nhắn
(6) Bảo tồn địa chỉ Internet
(7) Thông báo lỗi
(8) Bảo mật, tính toàn vẹn và xác thực
(9) Đàm phán nội dung

Chúng tôi dành một phần cho từng lĩnh vực, bao gồm động lực thay đổi và mô tả các tính năng mới tương ứng.

2. Khả năng mở rộng

Ngay từ đầu, nỗ lực HTTP=1.1 đã giả định rằng khả năng tương thích với cơ sở triển khai HTTP đã cài đặt (bao gồm nhiều triển khai không tuân thủ [2]) là bắt buộc. Có vẻ như hầu hết các nhà cung cấp phần mềm hoặc nhà điều hành trang web sẽ không triển khai các hệ thống không tương tác được với hàng triệu máy khách, máy chủ và proxy hiện có.

Vì nỗ lực HTTP=1.1 mất hơn bốn năm và tạo ra nhiều tài liệu dự thảo tạm thời nên nhiều người triển khai đã triển khai các hệ thống sử dụng phiên bản giao thức 'HTTP=1.1' trước khi phiên bản cuối cùng của đặc tả được hoàn thành. Điều này tạo ra một vấn đề tương thích khác: phiên bản cuối cùng phải tương thích cơ bản với các phiên bản pseudo-HTTP=1.1 này, ngay cả khi các bản nháp tạm thời hóa ra có lỗi trong đó.

Những yêu cầu tuyệt đối này về khả năng tương thích với các phiên bản trước được chỉ định kém đã dẫn đến một số đặc điểm riêng và sự không đồng nhất trong thiết kế cuối cùng. Không thể hiểu được cơ sở lý luận của tất cả các tính năng HTTP=1.1 nếu không nhận ra điểm này.

Vấn đề tương thích cũng nhấn mạnh sự cần thiết phải bao gồm, trong HTTP=1.1, càng nhiều hỗ trợ càng tốt cho khả năng mở rộng trong tương lai. Nghĩa là, nếu một phiên bản tương lai của HTTP được thiết kế thì nó sẽ không bị cản trở bởi bất kỳ vấn đề tương thích bổ sung nào.

Lưu ý rằng HTTP luôn chỉ định rằng nếu quá trình triển khai nhận được tiêu đề mà nó không hiểu thì nó phải bỏ qua tiêu đề đó. Quy tắc này cho phép vô số tiện ích mở rộng mà không có bất kỳ thay đổi nào đối với phiên bản giao thức, mặc dù bản thân nó không hỗ trợ tất cả các tiện ích mở rộng có thể có.

2.1. Số phiên bản

Bất chấp sự nhầm lẫn về ý nghĩa của mã thông báo phiên bản giao thức 'HTTP=1.1' (nó có hàm ý khả năng tương thích với một trong các bản nháp tạm thời hoặc với tiêu chuẩn cuối cùng không?), trong nhiều trường hợp, số phiên bản trong thông báo HTTP có thể được sử dụng để suy ra khả năng của người gửi. Tài liệu đồng hành với đặc tả HTTP [26] đã xác định rõ ràng các quy tắc cơ bản cho việc sử dụng và giải thích số phiên bản HTTP.

Số phiên bản trong tin nhắn HTTP đề cập đến người gửi tin nhắn theo từng chặng chứ không phải người gửi từ đầu đến cuối. Do đó, số phiên bản của bản tin trực tiếp hữu ích trong việc xác định khả năng ở cấp độ bản tin theo từng bước nhưng không hữu ích lắm trong việc xác định khả năng từ đầu đến cuối. Ví dụ: nếu máy chủ gốc HTTP=1.1 nhận được tin nhắn được chuyển tiếp bởi proxy HTTP=1.1, thì từ tin nhắn đó, máy chủ đó không thể biết liệu máy khách cuối cùng sử dụng HTTP=1.0 hay HTTP=1.1.

Vì lý do này, cũng như để hỗ trợ việc gỡ lỗi, HTTP=1.1 xác định tiêu đề Via mô tả đường dẫn theo sau là một tin nhắn được chuyển tiếp. Thông tin đường dẫn bao gồm số phiên bản HTTP của tất cả người gửi dọc theo đường dẫn và được ghi lại bởi mỗi người nhận kế tiếp. (Chỉ người gửi cuối cùng trong số nhiều người gửi HTTP=1.0 liên tiếp sẽ được liệt kê, vì proxy HTTP=1.0 sẽ không thêm thông tin vào tiêu đề Via .)

Machine Translated by Google

2.2. Phương pháp TÙY CHỌN

HTTP=1.1 giới thiệu phương thức OPTIONS , một cách để khách hàng tìm hiểu về khả năng của một máy chủ mà không thực sự yêu cầu tài nguyên. Vì Ví dụ: một proxy có thể xác minh rằng máy chủ tuân thủ với một phiên bản cụ thể của giao thức. Không may thay, ngữ nghĩa chính xác của phương pháp OPTIONS là chủ đề của một cuộc tranh luận căng thẳng và chưa được giải quyết, và chúng tôi tin rằng cơ chế này vẫn chưa hoàn thiện được chỉ định.

2.3. Nâng cấp lên các giao thức khác

Để dễ dàng triển khai các giao thức không tương thích trong tương lai, HTTP=1.1 bao gồm giao thức mới Nâng cấp tiêu đề yêu cầu. Bằng cách gửi bản nâng cấp tiêu đề, máy khách có thể thông báo cho máy chủ về tập hợp các giao thức mà nó hỗ trợ như một phương tiện giao tiếp thay thế. Máy chủ có thể chọn chuyển đổi giao thức, nhưng điều này không bắt buộc.

3. Bộ nhớ đệm

Các nhà phát triển web đã sớm nhận ra rằng bộ nhớ đệm của các phản hồi đều có thể thực hiện được và rất cao mong muốn. Bộ nhớ đệm có hiệu quả vì một số nguồn tài nguyên được nhiều người dùng yêu cầu thường xuyên hoặc được lặp lại bởi một người dùng nhất định. Bộ nhớ đệm được sử dụng ở hầu hết Trình duyệt web và trong nhiều máy chủ proxy; đôi khi chúng cũng được sử dụng kết hợp với một số máy chủ gốc nhất định. Các sản phẩm bộ nhớ đệm web, chẳng hạn như như công cụ bộ nhớ đệm của Cisco [4] và Lưu lượng truy cập của Inktomi Máy chủ [18] (kể tên hai), hiện là một ngành kinh doanh lớn.

Nhiều nhà nghiên cứu đã nghiên cứu hiệu quả của bộ nhớ đệm HTTP [20,6,1,17]. Bộ nhớ đệm được cải thiện độ trễ do người dùng nhận thấy bằng cách loại bỏ mạng giao tiếp với máy chủ gốc. Bộ nhớ đệm cũng giảm mức tiêu thụ băng thông bằng cách tránh truyền các gói mạng không cần thiết. Giảm mức tiêu thụ băng thông cũng gián tiếp giảm độ trễ cho các tương tác không được lưu vào bộ nhớ đệm, bằng cách giảm tắc nghẽn mạng. Cuối cùng, bộ nhớ đệm có thể làm giảm tải trên máy chủ gốc (và trên proxy trung gian), cải thiện hơn nữa độ trễ cho các tương tác không được lưu vào bộ nhớ đệm.

Một rủi ro với bộ nhớ đệm là cơ chế bộ nhớ đệm có thể không 'minh bạch về mặt ngữ nghĩa': đó là là, nó có thể trả về một phản hồi khác với những gì sẽ được trả lại bằng cách liên lạc trực tiếp với máy chủ gốc. Trong khi một số ứng dụng có thể chịu được phản hồi không minh bạch, nhiều ứng dụng Web (thương mại điện tử chẳng hạn) không thể.

3.1. Bộ nhớ đệm trong HTTP=1.0

HTTP=1.0 cung cấp cơ chế lưu vào bộ nhớ đệm đơn giản. Máy chủ gốc có thể đánh dấu phản hồi bằng cách sử dụng Tiêu đề hết hạn , với thời gian cho đến khi bộ đệm có thể trả về phản hồi mà không vi phạm tính minh bạch về mặt ngữ nghĩa. Hơn nữa, bộ đệm có thể kiểm tra tính hợp lệ hiện tại của phản hồi bằng cách sử dụng cái được gọi là một yêu cầu có điều kiện: nó có thể bao gồm tiêu đề If-Modi-fied-Since trong yêu cầu về tài nguyên, chỉ định giá trị được cung cấp trong phản hồi được lưu trong bộ nhớ cache Tiêu đề được sửa đổi lần cuối . Sau đó máy chủ có thể phản hồi bằng mã trạng thái 304 (Không sửa đổi), ngụ ý rằng mục nhập bộ đệm là hợp lệ hoặc nó có thể gửi một phản hồi 200 (OK) bình thường để thay thế mục nhập bộ đệm.

HTTP=1.0 cũng bao gồm một cơ chế, Pragma: tiêu đề không có bộ đệm, để khách hàng chỉ ra rằng một yêu cầu sẽ không được đáp ứng từ bộ đệm.

Cơ chế bộ nhớ đệm HTTP=1.0 hoạt động ở mức độ vừa phải, nhưng nó có nhiều thiếu sót về mặt khái niệm. Nó không cho phép máy chủ gốc hoặc máy khách đưa ra hướng dẫn đầy đủ và rõ ràng cho bộ nhớ đệm; do đó, nó phụ thuộc vào một nhóm các phương pháp phỏng đoán không được chỉ định rõ ràng. Điều này dẫn đến hai vấn đề: bộ nhớ đệm không chính xác của một số phản hồi lẽ ra không nên có. đã được lưu vào bộ nhớ đệm và không thể lưu vào bộ nhớ đệm một số phản hồi điều đó có thể đã được lưu trữ. Những nguyên nhân trước đây vấn đề ngữ nghĩa; cái sau gây ra hiệu suất các vấn đề.

3.2. Bộ nhớ đệm trong HTTP=1.1

HTTP=1.1 cố gắng làm rõ các khái niệm đằng sau bộ nhớ đệm và cung cấp các cơ chế giao thức rõ ràng và có thể mở rộng cho bộ nhớ đệm. Trong khi nó vẫn giữ nguyên thiết kế HTTP=1.0 cơ bản, nó tăng cường cả thiết kế đó với các tính năng mới và với các thông số kỹ thuật cẩn thận hơn về các tính năng hiện có.

Trong thuật ngữ HTTP=1.1, mục nhập bộ đệm là mới cho đến khi đạt đến thời điểm hết hạn, tại thời điểm đó nó trở nên cũ kỹ. Bộ nhớ đệm không cần phải loại bỏ nội dung cũ mục nhập, nhưng thông thường nó phải xác nhận lại nó bằng

Machine Translated by Google

máy chủ gốc trước khi trả lại nó để đáp ứng yêu cầu tiếp theo. Tuy nhiên, giao thức cho phép cả máy chủ gốc và máy khách người dùng cuối ghi đè quy tắc cơ bản này.

Trong HTTP=1.0, bộ nhớ đệm đã xác thực lại mục nhập bằng cách sử dụng tiêu đề If-Modified-Since . Tiêu đề này sử dụng dấu thời gian tuyệt đối với độ phân giải một giây, điều này có thể dẫn đến lỗi bộ đệm do lỗi đồng bộ hóa đồng hồ hoặc do thiếu độ phân giải. Do đó, HTTP=1.1 giới thiệu khái niệm tổng quát hơn về chuỗi trình xác thực bộ nhớ đệm mờ đục, được gọi là thẻ thực thể. Nếu hai phản hồi cho cùng một tài nguyên có cùng một thẻ thực thể thì chúng (theo đặc điểm kỹ thuật) phải giống hệt nhau. Vì thẻ thực thể không rõ ràng nên máy chủ gốc có thể sử dụng bất kỳ thông tin nào mà nó cho là cần thiết để xây dựng nó (chẳng hạn như dấu thời gian chi tiết hoặc con trỏ cơ sở dữ liệu nội bộ), miễn là nó đáp ứng yêu cầu về tính duy nhất. Khách hàng có thể so sánh các thẻ thực thể về sự bình đẳng, nhưng không thể thao túng chúng. Máy chủ HTTP=1.1 đính kèm thẻ thực thể vào phản hồi bằng tiêu đề ETag .

HTTP=1.1 bao gồm một số tiêu đề yêu cầu có điều kiện mới, ngoài If-Modified-Since. Cơ bản nhất là If-None-Match, cho phép máy khách hiển thị một hoặc nhiều thẻ thực thể từ các mục trong bộ đệm của nó cho một tài nguyên. Nếu không có giá trị nào trong số này khớp với giá trị thẻ thực thể hiện tại của tài nguyên thì máy chủ sẽ trả về phản hồi bình thường; nếu không, nó có thể trả về phản hồi 304 (Không được sửa đổi) với tiêu đề ETag cho biết mục nhập bộ đệm nào hiện hợp lệ. Lưu ý rằng cơ chế này cho phép máy chủ duyệt qua một tập hợp các phản hồi có thể có, trong khi cơ chế If-Modified-Since chỉ tạo ra lần truy cập bộ đệm nếu phản hồi gần đây nhất hợp lệ.

HTTP=1.1 cũng thêm các tiêu đề có điều kiện mới được gọi là If-Unmodified-Since và If-Match, tạo các dạng điều kiện tiên quyết khác cho các yêu cầu. Những điều kiện tiên quyết này rất hữu ích trong những tình huống phức tạp hơn; cụ thể, hãy xem phần thảo luận trong Phần 4.1 về yêu cầu phạm vi.

3.3. Tiêu đề kiểm soát bộ đệm

Để làm cho các yêu cầu về bộ đệm trở nên rõ ràng hơn, HTTP=1.1 thêm tiêu đề Kiểm soát bộ đệm mới, cho phép một bộ chỉ thị kiểm soát bộ đệm có thể mở rộng được truyền đi trong cả hai yêu cầu và lại.

tài trợ. Tập hợp được xác định bởi HTTP=1.1 khá lớn, vì vậy chúng tôi tập trung vào một số thành viên đáng chú ý.

Bởi vì dấu thời gian tuyệt đối trong tiêu đề HTTP=1.0 Expires có thể dẫn đến lỗi khi có độ lệch đồng hồ (và các quan sát cho thấy rằng độ lệch đồng hồ nghiêm trọng là phổ biến), HTTP=1.1 có thể sử dụng thời gian hết hạn tương đối, thông qua chỉ thị max-age . (Nó cũng giới thiệu tiêu đề Tuổi để bộ nhớ đệm có thể cho biết thời gian phản hồi đã được lưu trong bộ nhớ đệm trong suốt quá trình thực hiện.)

Bởi vì một số người dùng có các yêu cầu về quyền riêng tư giới hạn việc lưu vào bộ nhớ đệm ngoài nhu cầu về tính minh bạch về ngữ nghĩa, nên các chỉ thị riêng tư và không lưu trữ cho phép máy chủ và máy khách ngăn chặn việc lưu trữ một số hoặc tất cả phản hồi. Tuy nhiên, điều này không đảm bảo quyền riêng tư; chỉ có cơ chế mật mã mới có thể mang lại sự riêng tư thực sự.

Một số proxy chuyển đổi các phản hồi (ví dụ: để giảm độ phức tạp của hình ảnh trước khi truyền qua liên kết chậm [8]), nhưng do một số phản hồi không thể được chuyển đổi một cách mù quáng mà không làm mất thông tin nên lệnh không chuyển đổi có thể được sử dụng để ngăn chặn các chuyển đổi .

3.4. Tiêu đề Vary

Bộ nhớ đệm tìm mục nhập bộ nhớ đệm bằng cách sử dụng giá trị khóa trong thuật toán tra cứu. Mô hình bộ nhớ đệm đơn giản trong HTTP=1.0 chỉ sử dụng URL được yêu cầu làm khóa bộ đệm. Tuy nhiên, cơ chế đàm phán nội dung (được mô tả trong Phần 10) phá vỡ mô hình này vì phản hồi có thể khác nhau không chỉ dựa trên URL mà còn dựa trên một hoặc nhiều tiêu đề yêu cầu (chẳng hạn như Ngôn ngữ chấp nhận và Bộ ký tự chấp nhận) .

Để hỗ trợ lưu vào bộ nhớ đệm các phản hồi đã thương lượng và để có khả năng mở rộng trong tương lai, HTTP=1.1 bao gồm tiêu đề phản hồi Vary . Trường tiêu đề này chứa danh sách các trường tiêu đề yêu cầu lựa chọn có liên quan đã tham gia vào việc lựa chọn biến thể phản hồi.
Để sử dụng biến thể cụ thể của phản hồi được lưu trong bộ nhớ đệm để trả lời yêu cầu tiếp theo, việc chọn tiêu đề yêu cầu của yêu cầu mới phải khớp chính xác với yêu cầu ban đầu.

Cơ chế mở rộng đơn giản và tinh tế này hoạt động trong nhiều trường hợp đàm phán, nhưng nó không cho phép có nhiều thông tin trong bộ nhớ đệm. Ví dụ, về nguyên tắc, bộ đệm thông minh có thể nhận ra

Machine Translated by Google

rằng một giá trị tiêu đề yêu cầu tương thích với một giá trị tiêu đề khác mà không bằng nhau. Nỗ lực phát triển HTTP=1.1 bao gồm nỗ lực cung cấp cái gọi là 'thương lượng nội dung minh bạch' cho phép lưu trữ một số hoạt động tham gia tích cực, nhưng cuối cùng không có sự đồng thuận nào được phát triển và nỗ lực này [16,15] đã được tách khỏi HTTP=1.1 sự chỉ rõ.

4. Tối ưu hóa băng thông

Băng thông mạng hầu như luôn bị giới hạn.
Cả về bản chất đều làm trì hoãn việc truyền dữ liệu và thông qua độ trễ xếp hàng được thêm vào do tắc nghẽn, việc lãng phí băng thông sẽ làm tăng độ trễ.
HTTP=1.0 lãng phí băng thông theo nhiều cách mà HTTP=1.1 xử lý. Một ví dụ điển hình là máy chủ gửi toàn bộ tài nguyên (lớn) khi máy khách chỉ cần một phần nhỏ trong đó. Không có cách nào trong HTTP=1.0 để yêu cầu một phần đối tượng. Ngoài ra, băng thông có thể bị lãng phí theo hướng chuyển tiếp: nếu máy chủ HTTP=1.0 không thể chấp nhận các yêu cầu lớn, nó sẽ trả về mã lỗi sau khi băng thông đã được sử dụng hết. Điều còn thiếu là khả năng đàm phán với máy chủ và đảm bảo khả năng xử lý các yêu cầu đó trước khi gửi chúng.

4.1. Yêu cầu phạm vi

Một khách hàng có thể chỉ cần một phần tài nguyên. Ví dụ: nó có thể chỉ muốn hiển thị phần đầu của một tài liệu dài hoặc có thể muốn tiếp tục tải xuống một tệp sau khi quá trình truyền bị chấm dứt ở giữa luồng. Yêu cầu phạm vi HTTP=1.1 cho phép khách hàng yêu cầu các phần của tài nguyên. Mặc dù cơ chế phạm vi có thể mở rộng sang các đơn vị khác (chẳng hạn như các chương của tài liệu hoặc khung của phim), HTTP=1.1 chỉ hỗ trợ phạm vi byte. Máy khách thực hiện yêu cầu phạm vi bằng cách đưa tiêu đề Phạm vi vào yêu cầu của mình, chỉ định một hoặc nhiều phạm vi byte liền kề.
Máy chủ có thể bỏ qua tiêu đề Phạm vi hoặc có thể trả về một hoặc nhiều phạm vi trong phản hồi.

Nếu phản hồi chứa một phạm vi chứ không phải toàn bộ tài nguyên thì phản hồi đó sẽ mang mã trạng thái 206 (Nội dung một phần). Mã này ngăn không cho bộ đệm proxy HTTP=1.0 vô tình coi phản hồi là một bộ đệm đầy đủ và sau đó sử dụng nó làm bộ đệm lại được lưu trong bộ nhớ đệm. đáp ứng yêu cầu tiếp theo. Trong phản hồi lại phạm vi, tiêu đề Phạm vi nội dung cho biết độ lệch và độ dài của phạm vi được trả về và loại MIME multipart=byteranges mới cho phép truyền nhiều phạm vi trong một thông báo.

Yêu cầu phạm vi có thể được sử dụng theo nhiều cách khác nhau, chẳng hạn như:

(1) Để đọc phần đầu của hình ảnh, xác định hình dạng của nó và do đó thực hiện bố cục trang mà không tải toàn bộ hình ảnh.

(2) Để hoàn tất quá trình truyền phản hồi bị gián đoạn (do người dùng hoặc do lỗi mạng); nói cách khác, để chuyển đổi một phần mục nhập bộ đệm thành một phản hồi hoàn chỉnh.

(3) Để đọc phần đuôi của một vật thể đang phát triển.

Một số dạng yêu cầu phạm vi này liên quan đến các điều kiện của bộ đệm. Nghĩa là, phản hồi thích hợp có thể phụ thuộc vào tính hợp lệ của mục nhập bộ đệm của máy khách (nếu có).

Ví dụ: loại đầu tiên (lấy tiền tố của tài nguyên) có thể được thực hiện vô điều kiện hoặc có thể được thực hiện với tiêu đề If-None-Match ; điều sau ngụ ý rằng máy khách chỉ muốn phạm vi nếu đối tượng cơ bản đã thay đổi và nếu không sẽ sử dụng mục nhập bộ đệm của nó.

Mặt khác, loại yêu cầu thứ hai được thực hiện khi máy khách không có mục nhập bộ đệm bao gồm phạm vi mong muốn. Do đó, khách hàng chỉ muốn phạm vi nếu đối tượng cơ bản không thay đổi; nếu không, nó muốn có phản hồi đầy đủ.

Điều này có thể được thực hiện bằng cách trước tiên gửi một yêu cầu phạm vi có tiêu đề If-Match , sau đó lặp lại yêu cầu mà không có tiêu đề nào nếu yêu cầu đầu tiên không thành công. Tuy nhiên, vì đây là một hoạt động tối ưu hóa quan trọng nên HTTP=1.1 bao gồm tiêu đề If-Range , tiêu đề này thực hiện hiệu quả chuỗi đó trong một tương tác duy nhất.

Yêu cầu phạm vi ban đầu được đề xuất bởi Ari Luotonen và John Franks [13], sử dụng phần mở rộng cho cú pháp URL thay vì trường tiêu đề riêng biệt.
Tuy nhiên, cách tiếp cận này tỏ ra ít tổng quát hơn so với cách tiếp cận cuối cùng được sử dụng trong HTTP=1.1, đặc biệt đối với các yêu cầu có điều kiện.

4.2. Kỳ vọng và 100 (Tiếp theo)

Một số yêu cầu HTTP (ví dụ: PUT hoặc phương thức POST) mang nội dung yêu cầu, có thể là

Machine Translated by Google

dài tùy ý. Nếu máy chủ không sẵn sàng chấp nhận yêu cầu, có lẽ do xác thực không thành công, việc truyền một lượng yêu cầu lớn như vậy sẽ gây lãng phí băng thông.

HTTP=1.1 bao gồm mã trạng thái mới, 100 (Tiếp tục), để thông báo cho khách hàng rằng nội dung yêu cầu sẽ được truyền đi. Khi cơ chế này được sử dụng, trước tiên máy khách sẽ gửi các tiêu đề yêu cầu của mình, sau đó chờ phản hồi. Nếu phản hồi là mã lỗi, chẳng hạn như 401 (Không được phép), cho biết rằng máy chủ không cần đọc nội dung yêu cầu thì yêu cầu sẽ bị chấm dứt. Nếu phản hồi là 100 (Tiếp tục), thì máy khách có thể gửi nội dung yêu cầu khi biết rằng máy chủ sẽ chấp nhận nó.

Tuy nhiên, máy khách HTTP=1.0 không hiểu phản hồi 100 (Tiếp tục). Do đó, để kích hoạt việc sử dụng cơ chế này, khách hàng sẽ gửi tiêu đề Expect mới , với giá trị 100-continue.

(Tiêu đề Expect có thể được sử dụng cho các mục đích khác trong tương lai không được xác định trong HTTP=1.1.)

Bởi vì không phải tất cả các máy chủ đều sử dụng cơ chế này (tiêu đề Expect là sự bổ sung tương đối muộn cho HTTP=1.1 và các máy chủ 'HTTP=1.1' đầu tiên không triển khai cơ chế này), máy khách không được đợi vô thời hạn cho phản hồi 100 (Tiếp tục) trước khi gửi phần thân yêu cầu của nó. HTTP=1.1 chỉ định một số quy tắc phức tạp để tránh phải chờ đợi vô hạn hoặc lãng phí băng thông. Chúng tôi thiếu kinh nghiệm đầy đủ dựa trên việc triển khai đã triển khai để biết liệu thiết kế này có hoạt động hiệu quả hay không.

4.3. Nén

Một cách nổi tiếng để tiết kiệm băng thông là sử dụng tính năng nén dữ liệu. Trong khi hầu hết các định dạng hình ảnh (GIF, JPEG, MPEG) đều được nén trước thì nhiều loại dữ liệu khác được sử dụng trên Web thì không. Một nghiên cứu cho thấy rằng việc sử dụng tích cực tính năng nén bổ sung có thể tiết kiệm gần 40% số byte được gửi qua HTTP [25]. Mặc dù HTTP=1.0 bao gồm một số hỗ trợ cho việc nén nhưng nó không cung cấp đủ cơ chế để đàm phán sử dụng tính năng nén hoặc để phân biệt giữa nén đầu cuối và nén từng bước.

HTTP=1.1 tạo ra sự khác biệt giữa mã hóa nội dung, là mã hóa đầu cuối có thể có ở định dạng gốc của tài nguyên và mã hóa chuyển, luôn theo từng bước nhảy. Việc nén có thể được thực hiện dưới dạng mã hóa nội dung hoặc dưới dạng mã hóa truyền. Để hỗ trợ lựa chọn này và lựa chọn giữa các mã hóa nén hiện tại và tương lai khác nhau mà không phá vỡ tính tương thích với cơ sở đã cài đặt, HTTP=1.1 đã phải sửa đổi và mở rộng một cách cẩn thận các cơ chế đàm phán việc sử dụng mã hóa.

HTTP=1.0 bao gồm tiêu đề Mã hóa nội dung , cho biết (các) mã hóa nội dung từ đầu đến cuối được sử dụng cho tin nhắn; HTTP=1.1 thêm tiêu đề Mã hóa chuyển , cho biết (các) mã hóa chuyển từng bước được sử dụng cho một tin nhắn.

HTTP=1.1 (không giống như HTTP=1.0) chỉ định cẩn thận tiêu đề Accept-Encoding , được khách hàng sử dụng để cho biết nội dung mã hóa nào nó có thể xử lý và nội dung nào nó thích hơn. Một vấn đề phức tạp là nhu cầu hỗ trợ các máy khách 'robot' đang cố gắng tạo các bản sao tài nguyên của máy chủ gốc; một vấn đề khác là nhu cầu tương tác với các triển khai HTTP=1.0, trong đó Mã hóa chấp nhận được chỉ định kém.

HTTP=1.1 cũng bao gồm tiêu đề TE , cho phép khách hàng chỉ ra mã truyền nào được chấp nhận và mã nào được ưu tiên. Lưu ý rằng một mã chuyển giao quan trọng, Chunked, có chức năng đặc biệt (không liên quan đến nén) và sẽ được thảo luận thêm trong Phần 6.1.

5. Quản lý kết nối mạng

HTTP hầu như luôn sử dụng TCP làm giao thức truyền tải. TCP hoạt động tốt nhất với các kết nối có thời gian tồn tại lâu nhưng thiết kế HTTP ban đầu sử dụng kết nối TCP mới cho mỗi yêu cầu, do đó, mỗi yêu cầu đều phải chịu chi phí thiết lập kết nối TCP mới (ít nhất một lần khứ hồi trên mạng, cộng với một số gói trên cao). Vì hầu hết các tương tác trên Web đều ngắn (kích thước thông báo phản hồi trung bình là khoảng 4 Kbyte [25]), các kết nối TCP hiếm khi vượt qua vùng 'bắt đầu chậm' [19] và do đó không thể tối đa hóa việc sử dụng băng thông sẵn có của chúng.

Các trang web thường có hình ảnh được nhúng, đôi khi có nhiều hình ảnh và mỗi hình ảnh được truy xuất lại thông qua một yêu cầu HTTP riêng biệt. Việc sử dụng kết nối TCP mới cho mỗi lần truy xuất hình ảnh sẽ tuần tự hóa việc hiển thị toàn bộ trang theo độ trễ thiết lập kết nối cho tất cả các yêu cầu. Netscape

Machine Translated by Google

giới thiệu việc sử dụng các kết nối TCP song song để bù đắp cho việc xê-ri hóa này, nhưng khả năng tắc nghẽn gia tăng sẽ hạn chế tiện ích của việc này tiếp cận.

Để giải quyết những vấn đề này, Padmanabhan và Mogul [33] đã khuyến nghị sử dụng các kết nối liên tục và sắp xếp các yêu cầu một cách liên tục. sự liên quan.

5.1. Tiêu đề kết nối

Trước khi thảo luận về các kết nối liên tục, chúng ta giải quyết một vấn đề cơ bản hơn. Với việc sử dụng các proxy trung gian, HTTP tạo ra sự khác biệt giữa đường đi từ đầu đến cuối của một tin nhắn và kết nối từng bước thực tế giữa hai HTTP triển khai.

HTTP=1.1 giới thiệu khái niệm hop-by-hop tiêu đề: tiêu đề thư chỉ áp dụng cho một kết nối chứ không phải toàn bộ đường dẫn. (Ví dụ: chúng tôi đã mô tả từng bước Các tiêu đề Mã hóa chuyển giao và TE .) Việc sử dụng tiêu đề hop-by-hop tạo ra một vấn đề tiềm ẩn: nếu tiêu đề như vậy sẽ được chuyển tiếp bởi một proxy ngây thơ, nó có thể đánh lừa người nhận.

Do đó, HTTP=1.1 bao gồm Kết nối tiêu đề. Tiêu đề này liệt kê tất cả các tiêu đề từng bước một trong một tin nhắn, cho người nhận biết rằng những tiêu đề này tiêu đề phải được loại bỏ khỏi tin nhắn đó trước khi nó được chuyển tiếp. Cơ chế mở rộng này cho phép sự ra mắt trong tương lai của các tiêu đề hop-by-hop mới; người gửi không cần biết người nhận có hiểu một tiêu đề mới để ngăn chặn người nhận chuyển tiếp tiêu đề.

Vì proxy HTTP=1.0 không hiểu được Tuy nhiên, tiêu đề kết nối áp dụng HTTP=1.1 một quy tắc bổ sung. Nếu tiêu đề Kết nối được nhận lại trong thông báo HTTP=1.0 thì nó phải có đã được chuyển tiếp không chính xác bởi proxy HTTP=1.0. Do đó, tất cả các tiêu đề mà nó liệt kê cũng được chuyển tiếp không chính xác và phải được bỏ qua.

Tiêu đề Kết nối cũng có thể liệt kê các mã thông báo kết nối, không phải là tiêu đề mà là các cờ Boolean cho mỗi kết nối. Ví dụ: HTTP=1.1 xác định mã thông báo đóng để cho phép thiết bị ngang hàng biểu thị rằng nó không muốn sử dụng kết nối liên tục. Một lần nữa, cơ chế tiêu đề kết nối ngăn chặn các mã thông báo này được chuyển tiếp.

5.2. Kết nối liên tục

HTTP=1.0, ở dạng tài liệu, không cung cấp tầm nhìn cho các kết nối liên tục. Một số HTTP=1.0 tuy nhiên, việc triển khai sử dụng Keep-Alive tiêu đề (được mô tả trong [12]) để yêu cầu duy trì kết nối. Thiết kế này không tương tác với các proxy trung gian (xem phần 19.6.2 của [9]); HTTP=1.1 chỉ định một giải pháp tổng quát hơn.

Để thừa nhận những đặc tính mong muốn của chúng, HTTP=1.1 đặt các kết nối liên tục làm mặc định. Máy khách, máy chủ và proxy HTTP=1.1 giả định rằng kết nối sẽ được giữ mở sau khi truyền của một yêu cầu và phản hồi của nó. Giao thức cho phép việc triển khai đóng kết nối bất cứ lúc nào thời gian, để quản lý tài nguyên của nó, mặc dù nó tốt nhất chỉ làm như vậy sau khi kết thúc phản hồi.

Bởi vì việc triển khai có thể không muốn sử dụng kết nối liên tục nếu nó không thể mở rộng hiệu quả tới số lượng lớn các kết nối hoặc có thể muốn làm sạch chấm dứt một vì lý do quản lý tài nguyên, giao thức cho phép nó gửi Kết nối: đóng tiêu đề để thông báo cho người nhận rằng kết nối sẽ không được tái sử dụng.

5.3. Đường ống

Mặc dù HTTP=1.1 khuyến khích việc truyền tải của nhiều yêu cầu qua một kết nối TCP, mỗi yêu cầu vẫn phải được gửi trong một liên tục tin nhắn và máy chủ phải gửi phản hồi (trên kết nối đã cho) theo thứ tự mà nó nhận được các yêu cầu tương ứng. Tuy nhiên, khách hàng không cần chờ đợi để nhận được phản hồi cho một yêu cầu trước gửi một yêu cầu khác trên cùng một kết nối. TRONG Trên thực tế, một khách hàng có thể gửi một số lượng lớn tùy ý yêu cầu qua kết nối TCP trước khi nhận được bất kỳ phản hồi nào. Thực hành này, được gọi là đường ống, có thể cải thiện đáng kể hiệu suất [31]. Nó tránh được sự cần thiết phải chờ đợi các chuyến đi khứ hồi của mạng, và nó tận dụng tối đa giao thức TCP.

6. Truyền tin nhắn

Thông điệp HTTP có thể mang nội dung tùy ý chiều dài. Người nhận tin nhắn cần biết

Machine Translated by Google

nơi tin nhắn kết thúc. Người gửi có thể sử dụng tiêu đề Độ dài nội dung để cung cấp độ dài của nội dung. Tuy nhiên, nhiều phản hồi được tạo ra một cách linh hoạt bởi các quy trình CGI [3] và các cơ chế tương tự. Nếu không đệm toàn bộ phản hồi (điều này sẽ tăng thêm độ trễ), máy chủ không thể biết nó sẽ kéo dài bao lâu và không thể gửi tiêu đề Độ dài nội dung .

Khi không sử dụng kết nối liên tục, giải pháp rất đơn giản: máy chủ sẽ đóng kết nối.
Tùy chọn này có sẵn ở dạng HTTP=1.1, nhưng nó làm mất đi ưu điểm về hiệu suất của các kết nối liên tục.

6.1. Mã hóa chuyển Chunked

HTTP=1.1 giải quyết vấn đề phân định nội dung thư bằng cách giới thiệu mã hóa chuyển Chunked . Người gửi chia nội dung thư thành các đoạn có độ dài tùy ý và mỗi đoạn được gửi với độ dài được thêm vào trước; nó đánh dấu sự kết thúc của tin nhắn bằng một đoạn có độ dài bằng 0. Người gửi sử dụng tiêu đề Transfer-Encoding: chunked để báo hiệu việc sử dụng chunking.

Cơ chế này cho phép người gửi đệm các phần nhỏ của tin nhắn, thay vì toàn bộ tin nhắn mà không cần thêm nhiều sự phức tạp hoặc chi phí. Tất cả các triển khai HTTP=1.1 phải có khả năng nhận được các tin nhắn được phân đoạn.

Mã hóa chuyển Chunked giải quyết một vấn đề khác, không liên quan đến hiệu suất. Trong HTTP=1.0, nếu người gửi không bao gồm tiêu đề Độ dài nội dung thì người nhận không thể biết liệu thư có bị cắt bớt do sự cố truyền hay không. Sự không rõ ràng này dẫn đến lỗi, đặc biệt khi các phản hồi bị cắt ngắn được lưu trữ trong bộ nhớ đệm.

6.2. Xe kéo

Chunking giải quyết một vấn đề khác liên quan đến việc lưu vào bộ đệm tin nhắn phía người gửi. Một số trường tiêu đề, chẳng hạn như Content-MD5 (tổng kiểm tra mật mã trên nội dung thư), không thể được tính toán cho đến khi nội dung thư được tạo. Trong HTTP=1.0, việc sử dụng các trường tiêu đề như vậy yêu cầu người gửi phải đệm toàn bộ thư.

Trong HTTP=1.1, một thông báo được phân đoạn có thể bao gồm đoạn giới thiệu sau đoạn cuối cùng. Trailer đơn giản là một bộ của một hoặc nhiều trường tiêu đề. Bằng cách đặt chúng ở cuối tin nhắn, người gửi cho phép mình tính toán chúng sau khi tạo nội dung tin nhắn.

Người gửi thông báo cho người nhận về sự hiện diện của đoạn giới thiệu thư bằng cách bao gồm tiêu đề Đoạn giới thiệu , trong đó liệt kê tập hợp các tiêu đề được trì hoãn cho đến đoạn giới thiệu. Ví dụ: cảnh báo này cho phép trình duyệt tránh hiển thị tiền tố của phản hồi trước khi nhận được thông tin xác thực có trong đoạn giới thiệu.

HTTP=1.1 áp đặt một số điều kiện nhất định đối với việc sử dụng đoạn giới thiệu để ngăn chặn một số loại lỗi tương tác nhất định. Ví dụ: nếu máy chủ gửi một tin nhắn dài có đoạn giới thiệu tới proxy HTTP=1.1 đang chuyển tiếp phản hồi tới máy khách HTTP=1.0 thì proxy phải đệm toàn bộ thư hoặc bỏ đoạn giới thiệu. Thay vì nhấn mạnh rằng các proxy sẽ đệm các tin nhắn dài tùy ý, điều này là không khả thi, giao thức đặt ra các quy tắc sẽ ngăn không cho bất kỳ thông tin quan trọng nào trong đoạn giới thiệu (chẳng hạn như thông tin xác thực) bị mất do sự cố này.
Cụ thể, máy chủ không thể gửi đoạn giới thiệu trừ khi thông tin chứa trong đó hoàn toàn là tùy chọn hoặc khách hàng đã gửi tiêu đề TE: trailer , cho biết rằng nó sẵn sàng nhận đoạn giới thiệu (và ngầm định đệm toàn bộ phản hồi nếu nó đang chuyển tiếp tin nhắn đến máy khách HTTP=1.0).

6.3. Vấn đề về độ dài truyền

Một số cơ chế HTTP=1.1, chẳng hạn như Xác thực quyền truy cập thông báo (xem Phần 9.1), yêu cầu thỏa thuận từ đầu đến cuối về độ dài của nội dung thư; đây được gọi là độ dài thực thể. Mã hóa truyền từng chặng, chẳng hạn như nén hoặc phân đoạn, có thể tạm thời thay đổi độ dài truyền của tin nhắn. Trước khi sự khác biệt này được làm rõ, một số triển khai trước đó đã sử dụng tiêu đề Độ dài nội dung một cách bừa bãi.

Do đó, HTTP=1.1 đưa ra một bộ quy tắc dài để chỉ ra và suy ra độ dài thực thể của tin nhắn. Ví dụ: nếu sử dụng mã hóa truyền không nhận dạng (do đó độ dài truyền và độ dài thực thể khác nhau), người gửi hoàn toàn không được phép sử dụng tiêu đề Độ dài nội dung . Khi một phản hồi chứa nhiều phạm vi byte, sử dụng Kiểu nội dung: multipart=byteranges thì định dạng tự phân tách này sẽ xác định độ dài truyền.

Machine Translated by Google

7. Bảo tồn địa chỉ Internet

Các công ty và tổ chức sử dụng URL để quảng cáo về bản thân cũng như các sản phẩm và dịch vụ của họ.
Khi một URL xuất hiện trong một phương tiện không phải là Web, mọi người dường như thích các URL 'tên máy chủ thuần túy' hơn; tức là các URL không có bất kỳ cú pháp đường dẫn nào theo sau tên máy chủ. Chúng thường được gọi là 'URL ảo', nhưng bất chấp sự chê bai ngụ ý, khó có khả năng người dùng không theo chủ nghĩa thuần túy sẽ từ bỏ thông lệ này, điều này đã dẫn đến việc tiếp tục tạo ra số lượng lớn tên máy chủ.

Địa chỉ IP được nhiều người coi là nguồn tài nguyên khan hiếm (trong khi chờ quá trình chuyển đổi không chắc chắn sang IPv6 [7]). Hệ thống tên miền (DNS) cho phép nhiều tên máy chủ được liên kết với cùng một địa chỉ IP. Thật không may, vì các nhà thiết kế ban đầu của HTTP đã không lường trước được 'thảm họa thành công' mà họ đang kích hoạt, nên các yêu cầu HTTP=1.0 không chuyển được phần tên máy chủ của URL yêu cầu. Ví dụ: nếu người dùng đưa ra yêu cầu về tài nguyên tại URL http://example1.org/home.html, trình duyệt sẽ gửi thông báo có Dòng yêu cầu:

NHẬN /home.html HTTP/1.0

đến máy chủ tại example1.org. Điều này ngăn chặn việc liên kết tên máy chủ của máy chủ HTTP khác, chẳng hạn như exampleB.org với cùng một địa chỉ IP, vì máy chủ nhận được thông báo như vậy không thể cho biết thông báo đó dành cho máy chủ nào. Do đó, sự phổ biến của các URL ảo gây ra sự gia tăng việc phân bổ địa chỉ IP.

Nhóm Chỉ đạo Kỹ thuật Internet (IESG), cơ quan quản lý quy trình IETF, nhấn mạnh rằng HTTP=1.1 thực hiện các bước để cải thiện việc bảo tồn địa chỉ IP. Vì HTTP=1.1 phải tương tác với HTTP=1.0 nên nó không thể thay đổi định dạng của Dòng yêu cầu để bao gồm tên máy chủ của máy chủ. Thay vào đó, HTTP=1.1 yêu cầu các yêu cầu bao gồm tiêu đề Máy chủ , được đề xuất lần đầu tiên bởi John Franks [14], mang tên máy chủ. Điều này chuyển đổi ví dụ trên thành:

NHẬN /home.html HTTP/1.1
Máy chủ: example1.org

Nếu URL tham chiếu đến một cổng không phải cổng mặc định (cổng TCP 80), thì cổng này cũng được cung cấp trong tiêu đề Máy chủ .

Rõ ràng, vì máy khách HTTP=1.0 sẽ không gửi tiêu đề Máy chủ nên máy chủ HTTP=1.1 không thể đơn giản từ chối tất cả các tin nhắn mà không có chúng. Tuy nhiên, đặc tả HTTP=1.1 yêu cầu máy chủ HTTP=1.1 phải từ chối mọi thông báo HTTP=1.1 không chứa tiêu đề Máy chủ .

Mục đích của cơ chế tiêu đề Máy chủ và đặc biệt là yêu cầu thực thi sự hiện diện của nó trong các yêu cầu HTTP=1.1, là đẩy nhanh quá trình chuyển đổi khỏi việc chỉ định địa chỉ IP mới cho mọi URL ảo. Tuy nhiên, miễn là một phần đáng kể người dùng trên Internet sử dụng trình duyệt không gửi Máy chủ, thì không có nhà điều hành trang Web nào (chẳng hạn như doanh nghiệp thương mại điện tử) phụ thuộc vào những người dùng này sẽ từ bỏ địa chỉ IP vanity-URL. Do đó, quá trình chuyển đổi có thể mất nhiều năm. Nó có thể được loại bỏ bằng cách chuyển đổi sang IPv6 sớm hơn hoặc bằng cách sử dụng các cơ chế thị trường để ngăn cản việc sử dụng địa chỉ IPv4 không cần thiết.

8. Thông báo lỗi

HTTP=1.0 đã xác định một tập hợp tương đối nhỏ gồm 16 mã trạng thái, bao gồm mã 200 (OK) thông thường.
Kinh nghiệm cho thấy sự cần thiết phải có giải pháp tốt hơn trong việc báo cáo lỗi.

8.1. Tiêu đề cảnh báo

Mã trạng thái HTTP cho biết yêu cầu thành công hay thất bại. Để phản hồi thành công, mã trạng thái không thể cung cấp thông tin tư vấn bổ sung, một phần vì vị trí của mã trạng thái trong Dòng trạng thái, thay vì trong trường tiêu đề, sẽ ngăn việc sử dụng nhiều mã trạng thái.

HTTP=1.1 giới thiệu tiêu đề Cảnh báo , tiêu đề này có thể mang bất kỳ số lượng chỉ báo trạng thái phụ nào. Mục đích là cho phép người gửi thông báo cho người nhận rằng có điều gì đó không thỏa đáng về một phản hồi có vẻ thành công.

HTTP=1.1 xác định bộ mã Cảnh báo ban đầu , chủ yếu liên quan đến hành động của bộ đệm dọc theo đường dẫn phản hồi. Ví dụ: Cảnh báo có thể đánh dấu phản hồi là đã được bộ đệm trả về trong quá trình hoạt động bị ngắt kết nối, khi không thể xác thực mục nhập bộ đệm với máy chủ gốc.

Mã cảnh báo được chia thành hai loại,

Machine Translated by Google

dựa trên chữ số đầu tiên của mã 3 chữ số. Một loại cảnh báo phải bị xóa sau khi xác nhận lại thành công mục nhập bộ đệm; lớp còn lại phải được giữ lại với mục nhập bộ đệm được xác thực lại. Bởi vì sự khác biệt này được thực hiện dựa trên chữ số đầu tiên của mã, thay vì thông qua danh sách đầy đủ các mã, nên nó có thể mở rộng thành các mã Cảnh báo được xác định trong tương lai.

8.2. Mã trạng thái mới khác

Có 24 mã trạng thái mới trong HTTP=1.1; chúng tôi đã thảo luận về 100 (Tiếp tục), 206 (Nội dung một phần) và 300 (Nhiều lựa chọn) ở những phần khác trong bài viết này. Một số bổ sung đáng chú ý bao gồm: ž 409 (Xung đột), được trả về khi một yêu cầu xung đột với trạng thái hiện tại của tài nguyên. Ví dụ: yêu cầu PUT có thể vi phạm chính sách tạo phiên bản. ž 410 (Đã biến mất), được sử dụng khi tài nguyên đã được di chuyển lại vĩnh viễn khỏi máy chủ và để hỗ trợ xóa mọi liên kết đến tài nguyên.

Hầu hết các mã trạng thái mới khác đều là các tiện ích mở rộng nhỏ.

9. Bảo mật, tính toàn vẹn và xác thực

Trong những năm gần đây, IETF đã nâng cao sự nhạy cảm của mình đối với các vấn đề về quyền riêng tư và bảo mật. Một mối quan tâm đặc biệt là việc loại bỏ các mật khẩu được truyền đi một cách 'rõ ràng'. Sự nhấn mạnh ngày càng tăng này đã thể hiện trong đặc tả HTTP=1.1 (và các thông số kỹ thuật có liên quan chặt chẽ khác).

9.1. Xác thực truy cập thông báo

HTTP=1.0 cung cấp cơ chế kiểm soát truy cập phản hồi thử thách, xác thực cơ bản. Máy chủ gốc phản hồi yêu cầu cần xác thực bằng tiêu đề WWW-Authenticate xác định sơ đồ xác thực (trong trường hợp này là 'Cơ bản') và lĩnh vực. (Giá trị vùng cho phép máy chủ phân vùng các bộ tài nguyên thành 'không gian bảo vệ', mỗi bộ có cơ sở dữ liệu ủy quyền riêng.)

Máy khách (tác nhân người dùng) thường truy vấn người dùng về tên người dùng và mật khẩu cho khu vực, sau đó lặp lại yêu cầu ban đầu, lần này bao gồm cả Tiêu đề ủy quyền chứa tên người dùng và mật khẩu. Giả sử những thông tin xác thực này có thể được chấp nhận, máy chủ gốc sẽ phản hồi bằng cách gửi nội dung mong đợi. Khách hàng có thể tiếp tục gửi thông tin xác thực tương tự cho các tài nguyên khác trong cùng khu vực trên cùng một máy chủ, do đó loại bỏ chi phí bổ sung cho thử thách và phản hồi.

Một lỗ hổng nghiêm trọng trong xác thực cơ bản là tên người dùng và mật khẩu trong thông tin xác thực không được mã hóa và do đó dễ bị rình mò mạng. Thông tin đăng nhập cũng không phụ thuộc vào thời gian nên chúng có thể được thu thập thoải mái và sử dụng rất lâu sau khi được thu thập. Xác thực truy cập thông báo [10,11] cung cấp một cơ chế đơn giản sử dụng cùng một khung với xác thực Cơ bản đồng thời loại bỏ nhiều sai sót của nó. (Xác thực quyền truy cập thông báo, phần lớn có thể tách biệt khỏi đặc tả HTTP=1.1, đã được phát triển song song với nó.)

Luồng thông báo trong xác thực truy cập Digest phản ánh luồng thông báo của Basic và sử dụng cùng các tiêu đề nhưng có sơ đồ 'Thông báo'. Thử thách của máy chủ trong xác thực truy cập Digest sử dụng giá trị nonce (một lần), cùng với các thông tin khác. Để phản hồi thành công, khách hàng phải tính tổng kiểm tra (MD5, theo mặc định) của tên người dùng, mật khẩu, số không, phương thức HTTP của yêu cầu và URI được yêu cầu. Mật khẩu không chỉ không còn được mã hóa mà phản hồi đã cho chỉ đúng đối với một tài nguyên và phương thức duy nhất. Do đó, kẻ tấn công có thể rình mò mạng chỉ có thể phát lại yêu cầu, phản hồi mà hắn đã thấy. Không giống như xác thực Cơ bản, việc lấy được những thông tin xác thực này không cung cấp quyền truy cập vào các tài nguyên khác.

Giống như xác thực Cơ bản, máy khách có thể thực hiện thêm các yêu cầu tới cùng một lĩnh vực và bao gồm thông tin xác thực Di-gest, được tính toán bằng phương pháp yêu cầu lại và URI yêu cầu thích hợp. Tuy nhiên, giá trị nonce của máy chủ gốc có thể phụ thuộc vào thời gian. Máy chủ có thể từ chối thông tin xác thực bằng cách cho biết phản hồi lại đã sử dụng thông tin cũ và bằng cách cung cấp thông tin mới. Sau đó, khách hàng có thể tính toán lại thông tin đăng nhập của mình mà không cần hỏi lại tên người dùng và mật khẩu.

Ngoài khả năng xác thực đơn giản, xác thực truy cập Digest còn cung cấp hai tính năng khác: hỗ trợ máy chủ xác thực của bên thứ ba và tính năng toàn vẹn thông báo có giới hạn (thông qua tiêu đề Thông tin xác thực) .

Machine Translated by Google

9.2. Xác thực proxy

Một số máy chủ proxy chỉ cung cấp dịch vụ cho các máy khách được xác thực phù hợp. Ví dụ: điều này ngăn chặn các máy khách khác lấy cắp băng thông từ một proxy không nghi ngờ.

Để hỗ trợ xác thực proxy, HTTP=1.1 giới thiệu các tiêu đề Proxy-Authenticate và Proxy-Authorization . Chúng đóng vai trò tương tự như các tiêu đề WWW-Authenticate và Authorization trong HTTP=1.0, ngoại trừ các tiêu đề mới là hop-by-hop, thay vì end-to-end. Xác thực proxy có thể sử dụng một trong hai sơ đồ xác thực Digest hoặc Basic, nhưng sơ đồ xác thực trước được ưu tiên hơn.

Máy chủ proxy gửi cho khách hàng tiêu đề Xác thực proxy , chứa thử thách, trong phản hồi 407 (Yêu cầu xác thực proxy).
Sau đó, máy khách lặp lại yêu cầu ban đầu nhưng thêm tiêu đề Ủy quyền proxy chứa thông tin xác thực phù hợp với thử thách. Sau khi xác thực proxy thành công, máy khách thường gửi cùng một tiêu đề Ủy quyền proxy đến proxy với mỗi yêu cầu tiếp theo, thay vì đợi để được thử thách lại.

9.3. Bảo vệ quyền riêng tư của URI

URI của một tài nguyên thường thể hiện thông tin mà một số người dùng có thể xem là riêng tư. Người dùng có thể không muốn để mọi người biết rằng họ đã truy cập một số trang web nhất định.

Tiêu đề Người giới thiệu [sic] trong một yêu cầu cung cấp cho máy chủ URI của tài nguyên mà từ đó URI yêu cầu được lấy. Điều này cung cấp cho máy chủ thông tin về lượt xem trang trước đó của người dùng. Để bảo vệ khỏi các hành vi vi phạm quyền riêng tư không mong muốn, đặc tả HTTP=1.1 cố gắng ngăn chặn việc gửi tiêu đề Người giới thiệu một cách không thích hợp; ví dụ: khi người dùng nhập URL từ bàn phím, ứng dụng không được gửi tiêu đề Người giới thiệu mô tả trang hiện đang hiển thị và khách hàng cũng không nên gửi tiêu đề Người giới thiệu trong một yêu cầu lại không an toàn nếu trang giới thiệu đã được được chuyển giao một cách an toàn.

Việc sử dụng biểu mẫu HTML dựa trên GET gây ra việc mã hóa các tham số biểu mẫu trong URI yêu cầu.
Hầu hết các máy chủ proxy đều ghi lại các URI yêu cầu này. Để bảo vệ chống lại việc tiết lộ thông tin nhạy cảm, chẳng hạn như mật khẩu hoặc số thẻ tín dụng, trong URI, đặc tả HTTP=1.1 không khuyến khích sử dụng các biểu mẫu dựa trên GET để gửi dữ liệu đó. Việc sử dụng các biểu mẫu dựa trên POST sẽ ngăn các tham số biểu mẫu xuất hiện trong URI yêu cầu và do đó không bị ghi lại một cách không thích hợp.

9.4. Nội dung-MD5

Tiêu đề Content-MD5 của MIME chứa bản tóm tắt MD5 của thực thể được gửi [28]. Đặc tả HTTP=1.1 cung cấp các quy tắc cụ thể cho việc sử dụng tiêu đề này trên Web, khác một chút so với việc sử dụng tiêu đề này trong MIME (thư điện tử). Người gửi có thể chọn gửi Content-MD5 để người nhận có thể phát hiện những thay đổi ngẫu nhiên đối với thực thể trong quá trình truyền. Content-MD5 là một ví dụ điển hình về tiêu đề mà máy chủ có thể gửi dưới dạng đoạn giới thiệu một cách hữu ích.

Rõ ràng giá trị Content-MD5 có thể dễ dàng bị giả mạo và không thể dùng làm phương tiện bảo mật.
Ngoài ra, vì Content-MD5 chỉ bao gồm thực thể trong một thông báo nên không thể sử dụng nó để xác định xem liệu một phản hồi đầy đủ đã được tập hợp lại thành công từ một số phản hồi một phần (phạm vi) hay liệu các tiêu đề phản hồi có bị thay đổi hay không.

9,5. Quản lý nhà nước

Yêu cầu HTTP không có trạng thái. Nghĩa là, từ góc độ của máy chủ, mỗi yêu cầu thường có thể được coi là độc lập với bất kỳ yêu cầu nào khác. Tuy nhiên, đối với các ứng dụng Web, trạng thái đôi khi có thể hữu ích.
Ví dụ: một ứng dụng mua sắm muốn theo dõi những gì có trong 'giỏ hàng' của người dùng, vì nội dung của giỏ thay đổi theo chuỗi yêu cầu HTTP.

Netscape đã giới thiệu 'cookie' [29] trong phiên bản 1.1 của trình duyệt của họ như một cơ chế quản lý trạng thái. IETF sau đó đã chuẩn hóa các cookie trong RFC2109 [21]. (Đặc tả cookie là một ví dụ khác về cách HTTP có thể được mở rộng bằng một đặc tả riêng biệt mà không ảnh hưởng đến giao thức cốt lõi. Hỗ trợ cookie là tùy chọn trong máy chủ và tác nhân người dùng, mặc dù một số dịch vụ dựa trên Web sẽ không hoạt động khi không có chúng.)

Cơ chế cookie cơ bản rất đơn giản. Máy chủ gốc gửi một đoạn thông tin (trạng thái) tùy ý đến máy khách trong phản hồi của nó. Khách hàng chịu trách nhiệm

Machine Translated by Google

có thể lưu thông tin và gửi lại thông tin đó cùng với yêu cầu tiếp theo tới máy chủ gốc. Đặc tả ban đầu của RFC2109 và Netscape đã nới lỏng mô hình này để cookie có thể được trả về bất kỳ tập hợp máy chủ nào có liên quan, thay vì chỉ một máy chủ. Các thông số kỹ thuật cũng hạn chế các URI nào trên một máy chủ nhất định mà cookie có thể được trả về. Máy chủ có thể chỉ định thời gian tồn tại cho cookie, sau đó nó không còn được sử dụng nữa.

Cookie có cả ý nghĩa riêng tư và bảo mật. Vì nội dung của chúng mang tính tùy ý nên cookie có thể chứa thông tin nhạy cảm phụ thuộc vào ứng dụng.
Ví dụ: chúng có thể chứa số thẻ tín dụng, tên người dùng và mật khẩu hoặc thông tin cá nhân khác. Các ứng dụng gửi thông tin như vậy qua kết nối không được mã hóa khiến thông tin đó dễ bị theo dõi và cookie được lưu trữ trên hệ thống máy khách có thể tiết lộ thông tin nhạy cảm cho người dùng khác của (hoặc kẻ xâm nhập) máy khách đó.

RFC2109 đã gây tranh cãi, chủ yếu là do các hạn chế được đưa ra để bảo vệ quyền riêng tư. Có lẽ vấn đề gây tranh cãi nhất trong số này liên quan đến 'giao dịch không thể xác minh' và 'cookie của bên thứ ba'. Hãy xem xét kịch bản này.

(1) Người dùng truy cập http://www.example1.com/ nhà.html.

(2) Trang được trả về chứa thẻ IMG (hình ảnh) có tham chiếu đến http://ad.example.com/adv1.gif, một quảng cáo.

(3) Trình duyệt của người dùng tự động yêu cầu hình ảnh. Phản hồi bao gồm cookie từ ad.example.com.

(4) Người dùng truy cập http://www.exampleB.com/home.html.

(5) Trang được trả về chứa thẻ IMG có tham chiếu đến http://ad.example.com/adv2. gif.

(6) Trình duyệt của người dùng tự động yêu cầu hình ảnh, gửi cookie đã nhận trước đó đến ad.example.com trong quá trình này. Phản hồi bao gồm một cookie mới từ ad.example.com.

Những người ủng hộ quyền riêng tư và những người khác lo lắng rằng: ở bước 3, người dùng nhận được cookie ('bên thứ ba') từ ad.example.com, một trang web mà cô ấy thậm chí còn không biết mình sẽ truy cập (một ' giao dịch không thể xác minh'); và cookie đầu tiên đó sẽ được trả về ad.example. com trong yêu cầu hình ảnh thứ hai (bước 6).

Nếu tiêu đề Người giới thiệu được gửi cùng với mỗi yêu cầu về độ tuổi hình ảnh tới ad.example.com thì trang web đó có thể bắt đầu tích lũy hồ sơ về sở thích của người dùng từ những trang web cô ấy đã truy cập, tại đây http://www.example1. com/home.html và http://www.exam pleB.com/home.html. Một trang web quảng cáo như vậy có thể chọn những quảng cáo mà cô ấy có thể quan tâm. Mặc dù quá trình lập hồ sơ đó tương đối đơn giản nếu xét riêng lẻ, nhưng nó có thể trở nên cá nhân hơn nếu hồ sơ cũng có thể được gắn với một người thực cụ thể chứ không chỉ là một cá nhân. Ví dụ: điều này có thể xảy ra nếu người dùng thực hiện một số loại đăng ký tại www.example1.com.

RFC2109 đã tìm cách hạn chế những tác động nguy hại có thể có của cookie bằng cách yêu cầu tác nhân người dùng từ chối các cookie đến từ phản hồi đối với các giao dịch không thể xác minh. RFC2109 còn tuyên bố thêm rằng tác nhân người dùng có thể được cấu hình để chấp nhận các cookie như vậy, miễn là mặc định không chấp nhận chúng. Cài đặt mặc định này là nguồn gây lo ngại cho các mạng quảng cáo (trong ví dụ là các công ty điều hành các trang web như ad.example.com) có mô hình kinh doanh phụ thuộc vào cookie và hoạt động kinh doanh của họ phát triển mạnh mẽ trong khoảng thời gian từ khi thông số kỹ thuật cơ bản hoàn tất (tháng 7 năm 1996) và thời điểm nó xuất hiện dưới dạng RFC (tháng 2 năm 1997). RFC2109 đã trải qua quá trình sàng lọc thêm [22] để đáp lại các nhận xét, cả về chính trị lẫn kỹ thuật.

10. Đàm phán nội dung

Người dùng web nói nhiều ngôn ngữ và sử dụng nhiều bộ ký tự. Một số tài nguyên Web có sẵn ở nhiều biến thể khác nhau để đáp ứng tính đa dạng này.
HTTP=1.0 bao gồm khái niệm đàm phán nội dung, một cơ chế mà khách hàng có thể thông báo cho máy chủ (các) ngôn ngữ và=hoặc (các) bộ ký tự nào được người dùng chấp nhận.

Đàm phán nội dung đã được chứng minh là một lĩnh vực gây tranh cãi và khó hiểu. Một số khía cạnh thoạt nhìn có vẻ đơn giản hóa ra lại khá khó giải quyết. Ví dụ: mặc dù thông lệ hiện tại của IETF là nhấn mạnh vào việc ghi nhãn bộ ký tự rõ ràng trong tất cả các ngữ cảnh có liên quan, nhưng thông lệ HTTP hiện tại là sử dụng một bộ ký tự mặc định trong hầu hết các ngữ cảnh, nhưng không phải tất cả các triển khai đều chọn cùng một mặc định.

Machine Translated by Google

Việc sử dụng các giá trị mặc định không được gắn nhãn làm phức tạp thêm vấn đề quốc tế hóa Web.

HTTP=1.0 đã cung cấp một số tính năng để hỗ trợ đàm phán nội dung, nhưng RFC1945 [2] không bao giờ sử dụng thuật ngữ đó và dành ít hơn một trang cho các tính năng chuyên môn có liên quan. Đặc tả HTTP=1.1 chỉ định các tính năng này một cách cẩn thận hơn nhiều và giới thiệu một số khái niệm mới.

Mục tiêu của cơ chế đàm phán nội dung là chọn đại diện tốt nhất có sẵn của tài nguyên.
HTTP=1.1 cung cấp hai hình thức đàm phán nội dung trực giao, khác nhau ở nơi thực hiện lựa chọn:

(1) Trong đàm phán do máy chủ điều khiển , hình thức hoàn thiện hơn, máy khách sẽ gửi gợi ý về sở thích của người dùng tới máy chủ, sử dụng các tiêu đề như Ngôn ngữ chấp nhận, Bộ ký tự chấp nhận, v.v. Sau đó, máy chủ sẽ chọn cách thể hiện tốt nhất phù hợp với các ưu tiên được thể hiện trong các tiêu đề này.

(2) Trong đàm phán theo hướng tác nhân , khi máy khách yêu cầu một tài nguyên khác nhau, máy chủ sẽ trả lời bằng phản hồi 300 (Nhiều lựa chọn) chứa danh sách các biểu diễn có sẵn và mô tả về thuộc tính của từng biểu diễn (chẳng hạn như ngôn ngữ và ký tự của nó). bộ). Sau đó, khách hàng (đại lý) sẽ chọn một đại diện, tự động hoặc có sự can thiệp của người dùng và gửi lại yêu cầu, chỉ định biến thể đã chọn.

Mặc dù đặc tả HTTP=1.1 dành riêng tên tiêu đề Thay thế [16] để sử dụng trong thương lượng do tác nhân điều khiển, nhóm làm việc HTTP chưa bao giờ hoàn thành đặc tả của tiêu đề này và thương lượng do máy chủ điều khiển vẫn là hình thức duy nhất có thể sử dụng được.

Một số người dùng có thể nói được nhiều ngôn ngữ nhưng với mức độ trôi chảy khác nhau. Tương tự, một tài nguyên Web có thể có sẵn ở ngôn ngữ gốc và ở một số bản dịch có độ chính xác khác nhau.
HTTP giới thiệu việc sử dụng các giá trị chất lượng để thể hiện tầm quan trọng hoặc mức độ chấp nhận được của các tham số có thể thương lượng khác nhau. Giá trị chất lượng (hoặc qvalue) là số điểm cố định trong khoảng từ 0,0 đến 1,0. Ví dụ: một người nói tiếng Anh bản ngữ với một chút thông thạo tiếng Pháp và có thể áp đặt một người bạn văn phòng nói tiếng Đan Mạch, có thể định cấu hình trình duyệt để tạo các yêu cầu bao gồm

Ngôn ngữ chấp nhận: en, fr;q=0.5, da;q=0.1

Bởi vì cơ chế đàm phán nội dung cho phép các giá trị q và ký tự đại diện, đồng thời thể hiện các biến thể trên nhiều khía cạnh (ngôn ngữ, bộ ký tự, loại nội dung và mã hóa nội dung), việc lựa chọn tự động kết hợp biến thể 'có sẵn tốt nhất' có thể phức tạp và có thể tạo ra kết quả không mong muốn.
Những lựa chọn này có thể tương tác với bộ nhớ đệm theo những cách tinh tế; xem phần thảo luận ở Phần 3.4.

Đàm phán nội dung hứa hẹn sẽ là một khu vực màu mỡ cho sự phát triển giao thức bổ sung. Ví dụ: nhóm làm việc HTTP đã nhận ra tiện ích của đàm phán tự động liên quan đến các tính năng triển khai ứng dụng khách, chẳng hạn như kích thước màn hình, độ phân giải và độ sâu màu. IETF đã thành lập nhóm công tác Đàm phán nội dung để tiếp tục công việc trong
khu vực.

11. Đề xuất đang chờ xử lý

Mặc dù nhóm làm việc HTTP sẽ giải tán sau khi xuất bản đặc tả HTTP=1.1 nhưng vẫn có rất nhiều đề xuất đang chờ xử lý để cải thiện thêm. Ở đây chúng tôi phác họa một vài trong số những cái quan trọng hơn.

Đặc tả HTTP=1.1 nhấn mạnh vào khả năng mở rộng nhưng không thể giải quyết hoàn toàn vấn đề này. Mặc dù nó chưa đạt được trạng thái của một nhóm làm việc, nhưng đã có một nỗ lực cố gắng xác định một cơ chế mở rộng chung [32].

Như chúng tôi đã lưu ý trong Phần 10, nhóm làm việc Thương lượng Nội dung đang nghiên cứu các đề xuất để xác định rõ hơn việc đàm phán nội dung [16,15] và các thẻ tính năng.

Một số nhà nghiên cứu đã quan sát thấy rằng khi tài nguyên Web thay đổi (do đó làm mất hiệu lực các mục trong bộ đệm), chúng thường không thay đổi nhiều [6,25]. Điều này cho thấy rằng việc chỉ truyền những khác biệt (hoặc delta) giữa giá trị tài nguyên hiện tại và phản hồi được lưu trong bộ nhớ đệm, thay vì toàn bộ phản hồi mới, có thể tiết kiệm băng thông và thời gian. Hai người chúng tôi, cùng với một số người khác, đã đề xuất một phần mở rộng đơn giản thành HTTP=1.1 để hỗ trợ mã hóa delta [27].

Trong Web ngày nay, nội dung được chia sẻ rộng rãi nhưng chủ yếu chảy theo một hướng, từ máy chủ đến máy khách.
Web có thể trở thành một phương tiện để phân phối các bản cập nhật cho nội dung được chia sẻ. Nhóm làm việc về Phiên bản và Tác giả Phân phối Web Toàn cầu (WEB-DAV) của IETF đang trong quá trình xác định các phần mở rộng HTTP để thực hiện tầm nhìn này [34].

Machine Translated by Google

12. Quan sát

HTTP=1.1 khác với HTTP=1.0 theo nhiều cách, cả lớn lẫn nhỏ. Mặc dù nhiều thay đổi trong số này rõ ràng là tốt hơn nhưng phần mô tả pro-tocol đã dài gấp ba lần và nhiều tính năng mới đã được giới thiệu mà không có bất kỳ đánh giá thử nghiệm thực tế nào để hỗ trợ chúng. Đặc tả HTTP=1.1 cũng bao gồm nhiều điểm bất thường về khả năng tương thích với cơ sở triển khai HTTP=1.0 đã cài đặt.

Sự gia tăng độ phức tạp này làm phức tạp công việc của máy khách, máy chủ và đặc biệt là những người cố vấn triển khai bộ đệm proxy. Nó đã dẫn đến những tương tác không mong muốn giữa các tính năng và có thể sẽ dẫn đến những tương tác khác. Chúng tôi không mong đợi việc áp dụng HTTP=1.1 sẽ diễn ra hoàn toàn mà không gặp trục trặc. May mắn thay, nhiều điều khoản trong HTTP=1.1 về khả năng mở rộng sẽ đơn giản hóa việc đưa ra các sửa đổi trong tương lai.

Sự nhìn nhận

Chúng tôi xin cảm ơn các trọng tài ẩn danh đã đánh giá.

Người giới thiệu

[1] M. Abrams, CR Standridge, G. Abdulla, S. Williams và EA Fox, Proxy bộ đệm: Hạn chế và tiềm năng, trong: Proc. của Hội nghị World Wide Web Quốc tế, tháng 12 năm 1995, http://ei.cs.vt.edu/¾succeed/WWW4/WW W4.html.

[2] T. Berners-Lee, R. Fielding và H. Frystyck, Giao thức truyền siêu văn bản-HTTP=1.0, RFC 1945, Nhóm làm việc HTTP, tháng 5 năm 1998.

[3] CGI: Giao diện cổng chung, tháng 3 năm 1998, http://www .w3.org/CGI/.

[4] Công cụ bộ đệm của Cisco, http://www.cisco.com/warp/public/751 /bộ nhớ đệm.

[5] Kho lưu trữ danh sách gửi thư HTTP-WG, tháng 9–tháng 11 năm 1994–1998. http://www.ics.uci.edu/pub/ietf/http/hypermail/.

[6] F. Douglis, A. Feldmann, B. Krishnamurthy và J. Mogul, Tỷ lệ thay đổi và các số liệu khác: Một nghiên cứu trực tiếp về World Wide Web, trong: Proc. Triệu chứng USENIX. về Công nghệ và Hệ thống Internet, tháng 12 năm 1997, trang 147–158. http://www.usenix.org/events/usits97.

[7] S. Deering và R. Hinden, Giao thức Internet Phiên bản 6, RFC 1883, IETF, tháng 12 năm 1995.

[8] A. Fox, SD Gribble, EA Brewer và E. Amir, Thích ứng với sự thay đổi của mạng và khách hàng thông qua động lực theo yêu cầu chuyển mã, trong: Proc. ASPLOS VII, Cambridge, MA, tháng 10 năm 1996, trang 160–170.

[9] R. Fielding, J. Gettys, JC Mogul, H. Frystyk, L. Mas-inter, P. Leach và T. Berners-Lee, Giao thức truyền siêu văn bản-HTTP=1.1, Internet Draft Draft-ietf- http-v11-spec-rev-06.txt, Nhóm công tác HTTP, ngày 18 tháng 11 năm 1998, ftp://ftp.ietf.org/internet-drafts/draft-ietf-http-v11-spe c-rev-06. txt; sẽ được phát hành dưới dạng RFC.

[10] J. Franks, P. Hallam-Baker, J. Hostetler, P. Leach, A. Lu-otonen, E. Sink và L. Stewart, Phần mở rộng cho HTTP: Xác thực truy cập thông báo, RFC 2069, IETF, tháng 1 1997, ftp://ftp.ietf.org/internet-drafts/draft-ietf-http-authenti cation-03.txt; Công việc này đang được tiến hành.

[11] J. Franks, P. Hallam-Baker, J. Hostetler, S. Lawrence, P. Leach, A. Luotonen và L. Stewart, Xác thực HTTP: Xác thực truy cập cơ bản và tóm tắt, Internet Draft Draft-ietf- http-authentication-03.txt, IETF, tháng 9 năm 1998, ftp://ftp.ietf.org/internet-drafts/draft-ietf-http-authenti cation-03.txt, đây là công việc đang được tiến hành.

[12] RT Fielding, Ghi chú lưu giữ, ngày 10 tháng 10 năm 1995. http://w ww.ics.uci.edu/pub/ietf/http/hypermail/1995q4/0063.html.

[13] J. Franks và A. Luotonen, dãy Byte — đề xuất thông số kỹ thuật chính thức, ngày 17 tháng 5 năm 1995, http://www.ics.uci.edu/pub/ietf/htt p/hypermail/1995q2/0122.html.

[14] J. Franks, Hai đề xuất cho HTTP/2.0, Tháng 11, 16 1994. http:// www.ics.uci.edu/pub/ietf/http/hypermail/1994q 4/0019.html.

[15] K. Holtman và AH Mutz, Thuật toán chọn lọc biến thể từ xa HTTP-RVSA=1.0, RFC 2296, Nhóm làm việc HTTP, tháng 3 năm 1998.

[16] K. Holtman và AH Mutz, Đàm phán nội dung minh bạch trong HTTP, RFC 2295, Nhóm làm việc HTTP, tháng 3 năm 1998.

[17] A. Iyengar và J. Challenger, Cải thiện hiệu suất của máy chủ Web bằng cách lưu trữ dữ liệu động, trong: Proc. Triệu chứng USENIX. về Công nghệ và Hệ thống Internet, tháng 12 năm 1997, http://www.usenix.org/events/usits97.

[18] Máy chủ lưu lượng truy cập Inktomi, http://www.inktomi.com/products/traffic/.

[19] V. Jacobson, Kiểm soát và tránh tắc nghẽn, trong: Proc. 1988 Hội nghị chuyên đề SIGCOMM '88 về Kiến trúc và Giao thức Truyền thông, Stanford, CA, 1988, trang 314–329.

[20] TM Kroeger, DDE Long và JC Mogul, Khám phá các giới hạn của việc giảm độ trễ web từ bộ nhớ đệm và tìm nạp trước, trong: Hội nghị chuyên đề về Hệ thống và Công nghệ Internet, Hiệp hội USENIX, tháng 12 năm 1997, http://www.usenix.o rg/publications/library/proceedings/usits97/kroeger.html.

[21] DM Kristol và L. Montulli, Quản lý nhà nước HTTP, RFC 2109, Nhóm công tác HTTP, tháng 2 năm 1997.

[22] DM Kristol và L. Montulli, Quản lý nhà nước HTTP, Dự thảo Internet Draft-ietf-http-state-man-mec-10.txt, Nhóm làm việc HTTP, tháng 7 năm 1998, http://portal.research.bell-labs .com/¾dmk/cookie-3.6.txt; Công việc này đang được tiến hành.

[23] J. Marshall, HTTP thực sự dễ dàng thực hiện, http://www.jmarshall. com/easy/http/, 1997.

[24] L. Masinter, Báo cáo triển khai HTTP=1.1 tới Bản nháp

Machine Translated by Google

Tiêu chuẩn, ngày 30 tháng 9 năm 1998, http://www.ietf.org/IESG/ht tp1.1-implementations.txt.

[25] JC Mogul, F. Douglis, A. Feldmann và B. Krishna-murthy, Lợi ích tiềm năng của mã hóa delta và nén dữ liệu cho HTTP, trong: Proc. Hội nghị SIGCOMM '97, Cannes, Pháp, tháng 9 năm 1997, trang 181–194.

[26] JC Mogul, RT Fielding, J. Gettys và H. Frystyck Nielsen, Sử dụng và giải thích số phiên bản HTTP, RFC 2145, Nhóm công tác HTTP, tháng 5 năm 1997.

[27] J. Mogul, B. Krishnamurthy, F. Douglis, A. Feldmann, Y. Goland và A. van Hoff, Mã hóa Delta trong HTTP, Internet Draft Draft-mogul-http-delta-01.txt, IETF, tháng 2 1999. ftp://ftp.ietf.org/internet-drafts/draft-mogul-http-delta -01.txt; Công việc này đang được tiến hành.

[28] J. Myers và M. Rose, Trường tiêu đề Content-MD5, RFC 1864, IETF, tháng 10 năm 1995.

[29] Netscape, Cookie HTTP trạng thái máy khách liên tục, http://www .netscape.com/newsref/std/cookie_spec.html.

[30] H. Frystyck Nielsen và J. Gettys, Danh sách tính năng HTTP=1.1 Tóm tắt Báo cáo, tháng 7 năm 1998, http://www.w3.org/Protocols/ HTTP/Diễn đàn/Báo cáo/.

[31] H. Frystyck Nielsen, J. Gettys, A. Baird-Smith, E. Prud'hommeux, HW Lie và C. Lilley, Hiệu ứng hiệu suất mạng của HTTP=1.1, CSS1 và PNG, trong: Proc. SIGCOMM '97, Cannes, Pháp, tháng 9 năm 1997.

[32] H. Frystyck Nielsen, P. Leach và S. Lawrence, HTTP Khung mở rộng cho các tiện ích mở rộng bắt buộc và tùy chọn, Bản nháp Internet-frystyk-http-extensions-01.txt, IETF, tháng 11 năm 1998, ftp://ftp.ietf.org/internet-drafts/draft -frystyk-http-extensions-01.txt; Công việc này đang được tiến hành.

[33] VN Padmanabhan và JC Mogul, Cải thiện độ trễ của HTTP, Comp. Hệ thống mạng ISDN 28 (1/2) (1995) 25–35.

[34] J. Slein, F. Vitali, E. Whitehead và D. Durand, Các yêu cầu đối với Giao thức tạo phiên bản và tác giả phân tán cho World Wide Web, RFC 2291, WEBDAV Nhóm, tháng 2 năm 1998.

[35] K. Thompson, GJ Miller và R. Wilder, Internet diện rộng mô hình và đặc điểm lưu lượng, Mạng IEEE 11 (6) (1997) 19–23.

[36] K. Yap, Tổng quan kỹ thuật về thông số kỹ thuật HTTP=1.1 mới, trong: Proc. Kỹ thuật World Wide Web của Úc Hội nghị, Brisbane, Queensland, Úc, tháng 5 năm 1997, http://www.dstc.edu.au/aw3tc/papers/yap/.

Balachander Krishnamurthy là nhà nghiên cứu tại Phòng thí nghiệm AT&T–Nghiên cứu ở Florham Park, NJ và có thể liên hệ tại bala@re tìm kiếm.att.com.

Jeffrey C. Mogul đã nhận được SB từ Viện Massachusetts Công nghệ năm 1979, một MS từ Đại học Stanford vào năm 1980, và bằng Tiến sĩ của Khoa Khoa học Máy tính của Đại học Stanford vào năm 1986. Tiến sĩ Mogul là người tham gia tích cực vào cộng đồng Internet và là tác giả hoặc đồng tác giả của một số Tiêu chuẩn Internet; gần đây nhất, anh ấy đã đóng góp rất nhiều cho đặc tả HTTP=1.1. Từ năm 1986, ông là nhà nghiên cứu tại Compaq (trước đây là Kỹ thuật số) Phòng thí nghiệm nghiên cứu phương Tây, nghiên cứu các vấn đề về mạng và hệ điều hành cho các hệ thống máy tính hiệu suất cao và cải thiện hiệu suất của Internet và World Wide Web. Ông là thành viên của ACM, Sigma Xi, và CPSR, và là Chủ tịch Ủy ban Chương trình Mùa đông 1994 Hội nghị kỹ thuật USENIX và IEEE TCOS lần thứ sáu Hội thảo về các chủ đề nóng trong hệ điều hành.

Địa chỉ liên hệ: Công ty máy tính Compaq Phòng thí nghiệm Nghiên cứu Phương Tây, 250 Đại lộ Đại học, Palo Alto, California, 94301 (mogul@pa.dec.com)

David M. Kristol hiện là một Nhân viên kỹ thuật tại Bell Phòng thí nghiệm, Công nghệ Lucent, trong Tạp chí Khoa học Thông tin Re- Trung tâm tìm kiếm, nơi anh ấy hiện đang làm nghiên cứu về an ninh và điện tử thương mại trên Internet. trước- Ngoài ra, anh ấy còn nghiên cứu các thông số kỹ thuật chính thức cho các giao thức truyền thông. Trong sáu năm, ông đã chịu trách nhiệm lại về trình biên dịch C cho UNIX(r) System V với tư cách là thành viên của Tổ chức hỗ trợ Unix của Bell Labs và ông là hiệu trưởng nhà phát triển trình biên dịch System V ANSI C. Anh ấy đã tham gia Bell Phòng thí nghiệm năm 1981. Trước đó, Kristol làm việc tại Massachusetts Hiệp hội Máy tính, nơi ông đã phát triển phép đo phổ khối hệ thống dữ liệu và tại GenRad, Inc., nơi ông đã phát triển hệ thống tự động hệ thống thiết bị kiểm tra. Ông đã nhận được bằng BA và BSEE từ Đại học Pennsylvania, Philadelphia, MS và ME bằng cấp (về Toán ứng dụng) của Đại học Harvard.

Địa chỉ email của anh ấy là: dmk@bell-labs.com.